

lab 1 - nested loop join

Query

Query History

```
45
46 explain (analyze, buffers)
47 select c.first_name, c.last_name, r.rental_date
48 from customer c
49 join rental r on c.customer_id = r.customer_id
50 where c.customer_id = 1;
51
```

Data Output

Messages

Notifications

Showing rows: 1 to 14

Page No: 1

of 1

	QUERY PLAN
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=3.307..6.207 rows=32.00 loops=1)
2	Buffers: shared read=153
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=2.688..2.693 rows=1.00 loops=1)
4	Index Cond: (customer_id = 1)
5	Index Searches: 1
6	Buffers: shared read=3
7	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.613..3.499 rows=32.00 loops=1)
8	Filter: (customer_id = 1)
9	Rows Removed by Filter: 16012
10	Buffers: shared read=150
11	Planning:
12	Buffers: shared hit=121 read=7 dirtied=2
13	Planning Time: 5.293 ms
14	Execution Time: 6.277 ms

Trả lời:

- customer là outer table vì được đọc trước và chỉ trả về 1 khách hàng.
- rental là inner table vì PostgreSQL dùng customer_id của khách hàng đó để tìm các lần thuê tương ứng.
- PostgreSQL chọn nested loop vì outer table chỉ có 1 dòng. Dùng hash join phải tạo hash table cho cả một bảng, tốn công hơn so với tra cứu một khách hàng cụ thể.
- loops = 1 ở inner table vì outer table có đúng 1 dòng, nên PostgreSQL chỉ cần chạy tìm kiếm trong rental một lần.

lab 2 - hash join

The screenshot shows a PostgreSQL query editor with a query window and a data output window. The query window contains the following SQL code:

```
50 where c.customer_id = 1;
51
52 explain (analyze, buffers)
53 select f.title, l.name as language
54 from film f
55 join language l on f.language_id = l.language_id;
56
57
```

The data output window shows the query plan for the above query. The plan is as follows:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Hash Join	(cost=1.14..103.62 rows=1000 width=99)			(actual time=0.024..0.356 rows=1000.00 loops=1)		
2	Hash Cond: (f.language_id = l.language_id)						
3	Buffers: shared hit=89						
4	-> Seq Scan on film f	(cost=0.00..98.00 rows=1000 width=17)			(actual time=0.009..0.120 rows=1000.00 loops=1)		
5	Buffers: shared hit=88						
6	-> Hash	(cost=1.06..1.06 rows=6 width=88)			(actual time=0.012..0.012 rows=6.00 loops=1)		
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB						
8	Buffers: shared hit=1						
9	-> Seq Scan on language l	(cost=0.00..1.06 rows=6 width=88)			(actual time=0.004..0.004 rows=6.00 loops=1)		
10	Buffers: shared hit=1						
11	Planning Time: 0.097 ms						
12	Execution Time: 0.397 ms						

Trả lời:

- language thường là bảng được hash vì nó rất nhỏ. PostgreSQL tạo hash table từ bảng này, đây là build phase.
- film là bảng được quét để dò vào hash table, đây là probe phase.
- Xem dòng memory usage bên trong node hash để ghi kết quả thực tế của máy cậu.
- Khi đặt `work_mem = '64kb'`, với bảng language nhỏ thì gần như không thay đổi: hash table vẫn vừa RAM và thường có batches: 1. Nếu bảng được hash lớn hơn bộ nhớ, PostgreSQL sẽ chia thành nhiều batch và có thể phải dùng ổ đĩa, làm chậm truy vấn.

lab 3 - merge join

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
51
52 explain (analyze, buffers)
53 select f.title, l.name as language
54 from film f
55 join language l on f.language_id = l.language_id;
56
57 set enable_hashjoin = off;
```

Below the query, the 'Data Output' tab is active, displaying the 'QUERY PLAN' for the query. The plan shows a Hash Join operation with a cost of 110.50..193.39 and 4581 rows. The join condition is Hash Cond: (i.film_id = f.film_id). The buffers show shared hits of 113. The plan also shows the Seq Scan on inventory i and the Hash operation on film f.

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Hash Join	110.50..193.39	4581	23	0.266..1.162	4581.00	1
2	Hash Cond: (i.film_id = f.film_id)						
3	Buffers: shared hit=113						
4	-> Seq Scan on inventory i	0.00..70.81	4581	6	0.007..0.227	4581.00	1
5	Buffers: shared hit=25						
6	-> Hash	98.00..98.00	1000	19	0.245..0.245	1000.00	1
7	Buckets: 1024 Batches: 1 Memory Usage: 59kB						
8	Buffers: shared hit=88						
9	-> Seq Scan on film f	0.00..98.00	1000	19	0.004..0.121	1000.00	1
10	Buffers: shared hit=88						
11	Planning:						
12	Buffers: shared hit=6						
13	Planning Time: 0.209 ms						
14	Execution Time: 1.298 ms						

Trả lời:

- Ở truy vấn đầu, kế hoạch nên có merge join.
- Nếu thấy index scan, PostgreSQL đã lấy dữ liệu theo thứ tự film_id từ index, nên không cần sort; đây là cách tối ưu hơn.
- Nếu thấy sort, nghĩa là ít nhất một bảng chưa có dữ liệu đúng thứ tự cần thiết, nên PostgreSQL phải tự sắp xếp trước khi merge.
- Sau reset all và bỏ order by, PostgreSQL thường chuyển lại hash join, vì join toàn bộ bảng bằng điều kiện = và không còn cần kết quả được sắp xếp. Tuy nhiên đây không phải quy tắc tuyệt đối: PostgreSQL sẽ chọn phương án có cost thấp nhất theo dữ liệu và statistics hiện tại.

